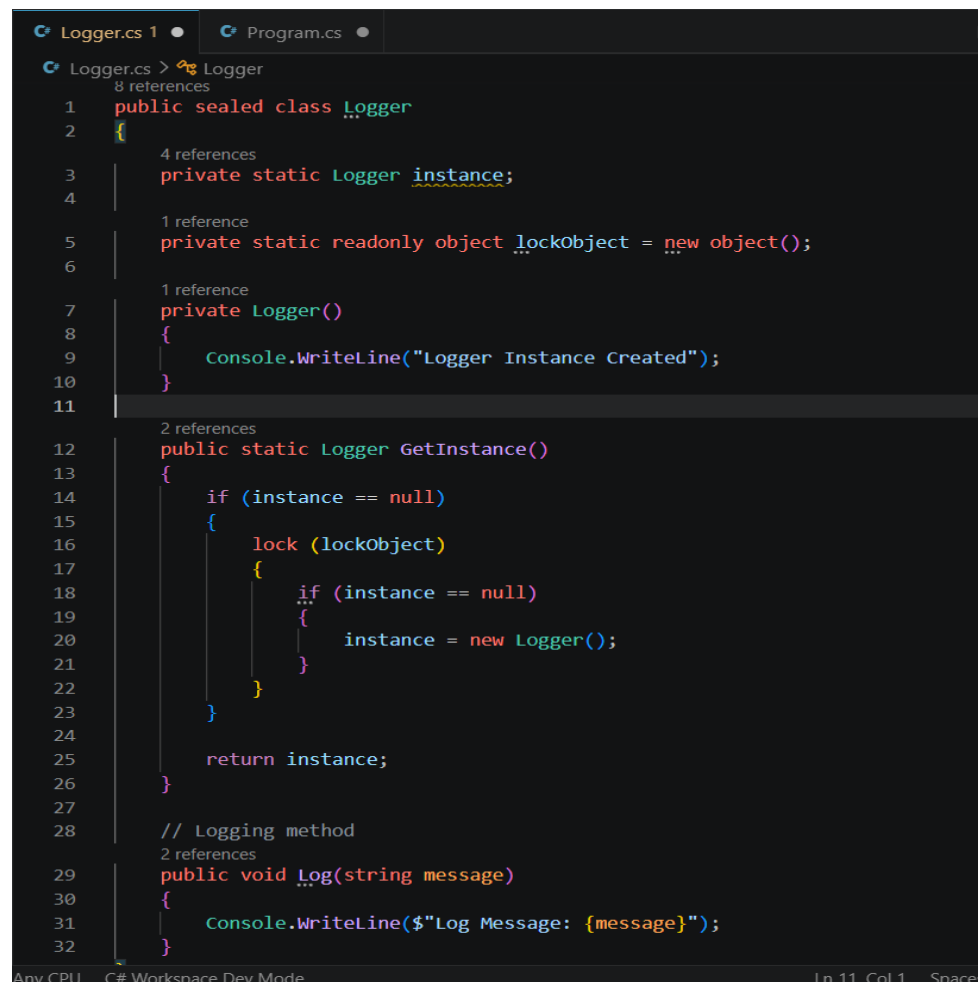


Exercise 1: Implementing the Singleton Pattern

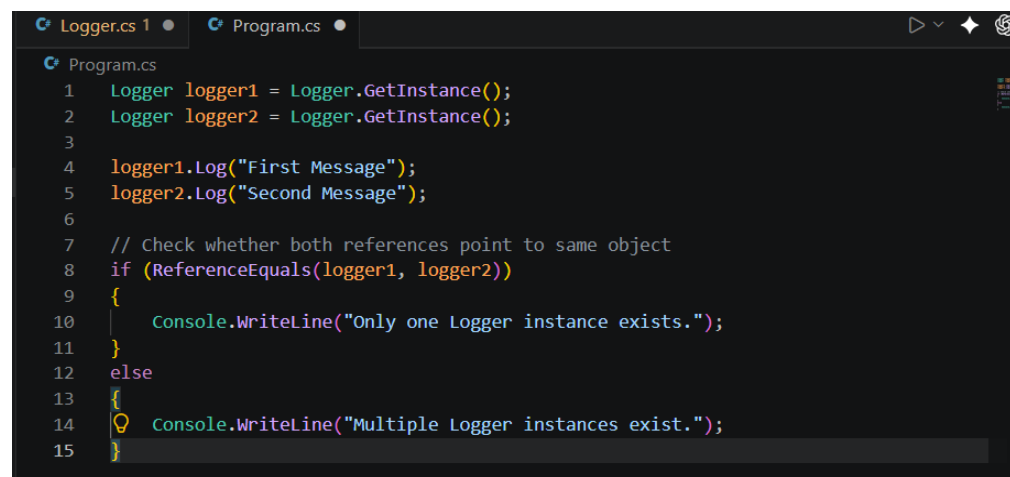
Scenario:

You need to ensure that a logging utility class in your application has only one instance throughout the application lifecycle to ensure consistent logging.

Code:



```
1 public sealed class Logger
2 {
3     private static Logger instance;
4
5     private static readonly object lockObject = new object();
6
7     private Logger()
8     {
9         Console.WriteLine("Logger Instance Created");
10    }
11
12    public static Logger GetInstance()
13    {
14        if (instance == null)
15        {
16            lock (lockObject)
17            {
18                if (instance == null)
19                {
20                    instance = new Logger();
21                }
22            }
23        }
24
25        return instance;
26    }
27
28    // Logging method
29    public void Log(string message)
30    {
31        Console.WriteLine($"Log Message: {message}");
32    }
```



```
1 Logger logger1 = Logger.GetInstance();
2 Logger logger2 = Logger.GetInstance();
3
4 logger1.Log("First Message");
5 logger2.Log("Second Message");
6
7 // Check whether both references point to same object
8 if (ReferenceEquals(logger1, logger2))
9 {
10    Console.WriteLine("Only one Logger instance exists.");
11 }
12 else
13 {
14    Console.WriteLine("Multiple Logger instances exist.");
15 }
```

Output:

```
PROBLEMS 1 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [ ] ... | [ ] [ ] X
PS C:\Users\nagaa\OneDrive\Desktop\upski\ConsoleApp1\SingletonPatternExample> dotnet run
Hello, World!
PS C:\Users\nagaa\OneDrive\Desktop\upski\ConsoleApp1\SingletonPatternExample> dotnet run
C:\Users\nagaa\OneDrive\Desktop\upski\ConsoleApp1\SingletonPatternExample\Logger.cs(4,27): warning CS8618
: Non-nullable field 'instance' must contain a non-null value when exiting constructor. Consider adding t
he 'required' modifier or declaring the field as nullable.
Logger Instance Created
Log Message: First Message
Log Message: Second Message
Only one Logger instance exists.
```

Exercise 2: Implementing the Factory Method Pattern

Scenario:

You are developing a document management system that needs to create different types of documents (e.g., Word, PDF, Excel). Use the Factory Method Pattern to achieve this.

CODE:

```
DocumentFactory.cs WordFactory.cs PdfFactory.cs ExcelFactory.cs
Program.cs
1 DocumentFactory wordFactory = new WordFactory();
2 IDocument wordDoc = wordFactory.CreateDocument();
3 wordDoc.Open();
4
5 DocumentFactory pdfFactory = new PdfFactory();
6 IDocument pdfDoc = pdfFactory.CreateDocument();
7 pdfDoc.Open();
8
9 DocumentFactory excelFactory = new ExcelFactory();
10 IDocument excelDoc = excelFactory.CreateDocument();
11 excelDoc.Open();
```

OUTPUT:

```
PS C:\Users\nagaa\OneDrive\Desktop\upski\ConsoleApp1\SingletonPatternExample\FactoryMethodPatternExample> dotnet run
Word Document Opened
PDF Document Opened
Excel Document Opened
PS C:\Users\nagaa\OneDrive\Desktop\upski\ConsoleApp1\SingletonPatternExample\FactoryMethodPatternExample>
```

Exercise 3: Implementing the Builder Pattern

Scenario:

You are developing a system to create complex objects such as a Computer with multiple optional parts. Use the Builder Pattern to manage the construction process.

Code:



```
Computer.cs 1 Program.cs X
Program.cs
1 Computer gamingPC = new Computer.Builder()
2   .SetCPU("Intel i9")
3   .SetRAM(32)
4   .SetStorage(1000)
5   .Build();
6
7 gamingPC.Display();
8
9 Computer officePC = new Computer.Builder()
10  .SetCPU("Intel i5")
11  .SetRAM(16)
12  .SetStorage(512)
13  .Build();
14
15 officePC.Display();

Computer.cs 1 X Program.cs
Computer.cs > Computer
1 public class Computer
25
26 public class Builder
27 {
28     2 references
29     public string CPU { get; private set; }
30     2 references
31     public int RAM { get; private set; }
32     2 references
33     public int Storage { get; private set; }
34
35     2 references
36     public Builder SetCPU(string cpu)
37     {
38         CPU = cpu;
39         return this;
40     }
41
42     2 references
43     public Builder SetRAM(int ram)
44     {
45         RAM = ram;
46         return this;
47     }
48
49     2 references
50     public Builder SetStorage(int storage)
51     {
52         Storage = storage;
53         return this;
54     }
55
56     2 references
57     public Computer Build()
58     {
59         return new Computer(this);
60     }
61 }
```

Output:

```
PS C:\Users\nagaa\OneDrive\Desktop\upski\BuilderPatternExample> dotnet run
Computer Configuration:
CPU      : Intel i9
RAM      : 32 GB
Storage  : 1000 GB

Computer Configuration:
CPU      : Intel i5
RAM      : 16 GB
Storage  : 512 GB

PS C:\Users\nagaa\OneDrive\Desktop\upski\BuilderPatternExample>
```

Exercise 4: Implementing the Adapter Pattern

Scenario:

You are developing a payment processing system that needs to integrate with multiple third-party payment gateways with different interfaces. Use the Adapter Pattern to achieve this.

Code:

```
PayPalGateway.cs  StripeGateway.cs  PayPalAdapter.cs  StripeAdapter.cs  Program.cs X
Program.cs
1  PaymentProcessor paypal =
2      new PayPalAdapter(new PayPalGateway());
3
4  paypal.ProcessPayment(1000);
5
6  PaymentProcessor stripe =
7      new StripeAdapter(new StripeGateway());
8
9  stripe.ProcessPayment(2000);
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\nagaa\OneDrive\Desktop\upski\AdapterPatternExample> dotnet run
Payment of Rs.1000 processed through PayPal.
Payment of Rs.2000 processed through Stripe.
PS C:\Users\nagaa\OneDrive\Desktop\upski\AdapterPatternExample>
```

Exercise 5: Implementing the Decorator Pattern

Scenario:

You are developing a notification system where notifications can be sent via multiple channels (e.g., Email, SMS). Use the Decorator Pattern to add functionalities dynamically.

Code:

```
NotifierDecorator.cs SMSNotifierDecorator.cs SlackNotifierDecorator.cs Program.cs X
Program.cs
1  Notifier notifier = new EmailNotifier();
2
3  Notifier smsNotifier =
4      new SMSNotifierDecorator(notifier);
5
6  Notifier slackAndSmsNotifier =
7      new SlackNotifierDecorator(smsNotifier);
8
9  slackAndSmsNotifier.Send("Your order has been shipped!");
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + v
PS C:\Users\nagaa\OneDrive\Desktop\upski\DecoratorPatternExample> dotnet clean
>> dotnet build
>> dotnet run

Build succeeded in 0.9s
Restore complete (0.7s)
  DecoratorPatternExample net10.0 succeeded (1.8s) -> bin\Debug\net10.0\DecoratorPatternExample.dll

Build succeeded in 3.6s
Email Notification: Your order has been shipped!
SMS Notification: Your order has been shipped!
Slack Notification: Your order has been shipped!
PS C:\Users\nagaa\OneDrive\Desktop\upski\DecoratorPatternExample>
```

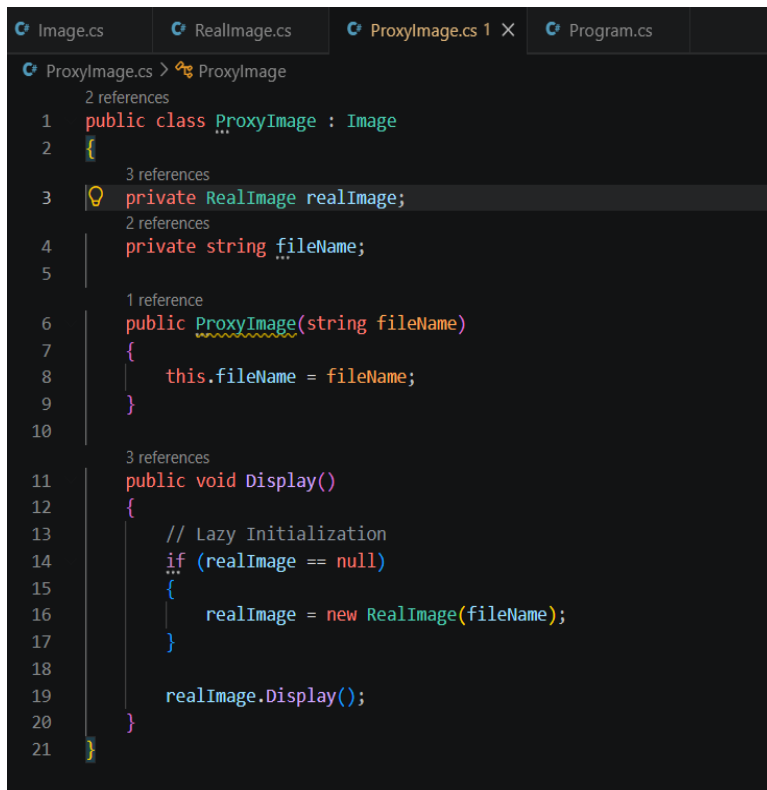
Exercise 6: Implementing the Proxy Pattern

Scenario:

You are developing an image viewer application that loads images from a remote server. Use the Proxy Pattern to add lazy initialization and caching.

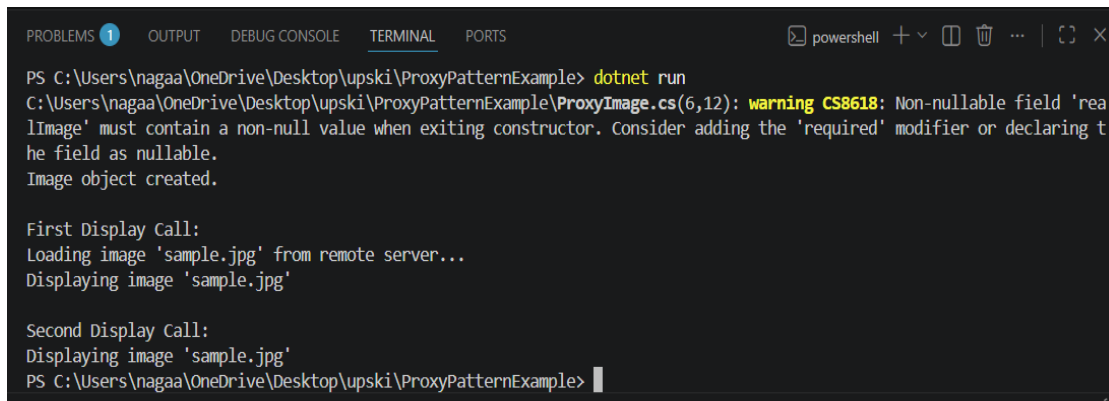
Code:

```
Image.cs RealImage.cs ProxyImage.cs 1 Program.cs X
Program.cs
1  Image image = new ProxyImage("sample.jpg");
2
3  Console.WriteLine("Image object created.");
4
5  Console.WriteLine("\nFirst Display Call:");
6  image.Display();
7
8  Console.WriteLine("\nSecond Display Call:");
9  image.Display();
```



```
1 public class ProxyImage : Image
2 {
3     private RealImage realImage;
4     private string fileName;
5
6     public ProxyImage(string fileName)
7     {
8         this.fileName = fileName;
9     }
10
11    public void Display()
12    {
13        // Lazy Initialization
14        if (realImage == null)
15        {
16            realImage = new RealImage(fileName);
17        }
18        realImage.Display();
19    }
20 }
21 }
```

Output:



```
PS C:\Users\nagaa\OneDrive\Desktop\upski\ProxyPatternExample> dotnet run
C:\Users\nagaa\OneDrive\Desktop\upski\ProxyPatternExample\ProxyImage.cs(6,12): warning CS8618: Non-nullable field 'realImage' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.
Image object created.

First Display Call:
Loading image 'sample.jpg' from remote server...
Displaying image 'sample.jpg'

Second Display Call:
Displaying image 'sample.jpg'
PS C:\Users\nagaa\OneDrive\Desktop\upski\ProxyPatternExample>
```

Exercise 7: Implementing the Observer Pattern

Scenario:

You are developing a stock market monitoring application where multiple clients need to be notified whenever stock prices change. Use the Observer Pattern to achieve this.

Code:

```

Program.cs
1  StockMarket stockMarket = new StockMarket("TCS");
2
3  Observer mobileUser = new MobileApp("Akshitha");
4  Observer webUser = new WebApp("Naga");
5
6  stockMarket.RegisterObserver(mobileUser);
7  stockMarket.RegisterObserver(webUser);
8
9  stockMarket.SetStockPrice(3500);
10
11 stockMarket.SetStockPrice(3700);
12
13 stockMarket.DeregisterObserver(webUser);
14
15 stockMarket.SetStockPrice(3900);

```

```

Observer.cs  Stock.cs  StockMarket.cs  MobileApp.cs  WebApp.cs  Program.cs
StockMarket.cs
3  public class StockMarket : Stock
9
10     1 reference
11     public StockMarket(string stockName)
12     {
13         this.stockName = stockName;
14     }
15
16     3 references
17     public void RegisterObserver(Observer observer)
18     {
19         observers.Add(observer);
20
21     2 references
22     public void DeregisterObserver(Observer observer)
23     {
24         observers.Remove(observer);
25
26     2 references
27     public void NotifyObservers()
28     {
29         foreach (Observer observer in observers)
30         {
31             observer.Update(stockName, stockPrice);
32         }
33
34     3 references
35     public void SetStockPrice(double price)
36     {
37         stockPrice = price;
38         Console.WriteLine($"{stockName} Stock Price Updated: {stockName} = Rs.{stockPrice}");
39     }

```

Mobile App [Akshitha] Notification: TCS price is Rs.3500
 Web App [Naga] Notification: TCS price is Rs.3500

any CPU C# Workspace Dev Mode Ln 39, Col 2 Spaces

Output:

```

PS C:\Users\nagaa\OneDrive\Desktop\upski\ObserverPatternExample> dotnet run

Stock Price Updated: TCS = Rs.3500
Mobile App [Akshitha] Notification: TCS price is Rs.3500
Web App [Naga] Notification: TCS price is Rs.3500

Stock Price Updated: TCS = Rs.3700
Mobile App [Akshitha] Notification: TCS price is Rs.3700
Web App [Naga] Notification: TCS price is Rs.3700

Stock Price Updated: TCS = Rs.3900
Mobile App [Akshitha] Notification: TCS price is Rs.3900
PS C:\Users\nagaa\OneDrive\Desktop\upski\ObserverPatternExample>

```

Exercise 8: Implementing the Strategy Pattern

Scenario:

You are developing a payment system where different payment methods (e.g., Credit Card, PayPal) can be selected at runtime. Use the Strategy Pattern to achieve this.

CODE:


```
CreditCardPayment.cs PayPalPayment.cs PaymentContext.cs 1 Program.cs X
1 PaymentContext paymentContext = new PaymentContext();
2
3 // Credit Card Payment
4 paymentContext.SetPaymentStrategy(new CreditCardPayment());
5 paymentContext.ExecutePayment(5000);
6
7 // PayPal Payment
8 paymentContext.SetPaymentStrategy(new PayPalPayment());
9 paymentContext.ExecutePayment(2500);
```

```
CreditCardPayment.cs PayPalPayment.cs PaymentContext.cs 1 X Program. D
PaymentContext.cs > PaymentContext
2 references
1 public class PaymentContext
2 {
3     2 references
4     private PaymentStrategy paymentStrategy;
5
6     2 references
7     public void SetPaymentStrategy(PaymentStrategy paymentStrategy)
8     {
9         this.paymentStrategy = paymentStrategy;
10    }
11
12    2 references
13    public void ExecutePayment(double amount)
14    {
15        paymentStrategy.Pay(amount);
16    }
17 }
```

```
CreditCardPayment.cs PayPalPayment.cs X PaymentContext.cs 1 Program. D
PayPalPayment.cs > PayPalPayment
1 reference
1 public class PayPalPayment : PaymentStrategy
2 {
3     2 references
4     public void Pay(double amount)
5     {
6         Console.WriteLine($"Paid Rs.{amount} using PayPal.");
7     }
8 }
```

OUTPUT:

```

PS C:\Users\nagaa\OneDrive\Desktop\upski\StrategyPatternExample> dotnet run
C:\Users\nagaa\OneDrive\Desktop\upski\StrategyPatternExample> PaymentContext.cs(3,29): warning CS8618: Non-nullable field 'paymentStrategy' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.
Paid Rs.5000 using Credit Card.
Paid Rs.2500 using PayPal.
PS C:\Users\nagaa\OneDrive\Desktop\upski\StrategyPatternExample>

```

Exercise 9: Implementing the Command Pattern

Scenario: You are developing a home automation system where commands can be issued to turn devices on or off. Use the Command Pattern to achieve this.

Code:

```
LightOnCommand.cs  LightOffCommand.cs  RemoteControl.cs 1  Program.cs X
Program.cs
1  Light light = new Light();
2
3  Command lightOn = new LightOnCommand(light);
4  Command lightOff = new LightOffCommand(light);
5
6  RemoteControl remote = new RemoteControl();
7
8  // Turn ON the light
9  remote.SetCommand(lightOn);
10 remote.PressButton();
11
12 // Turn OFF the light
13 remote.SetCommand(lightOff);
14 remote.PressButton();
```

```
LightOnCommand.cs  LightOffCommand.cs  RemoteControl.cs 1 X  C#

C# RemoteControl.cs > RemoteControl
2 references
1  public class RemoteControl
2  {
3      2 references
      private Command command;
4
5      2 references
      public void SetCommand(Command command)
6      {
7          this.command = command;
8      }
9
10     2 references
    public void PressButton()
11     {
12         command.Execute();
13     }
14 }
```

Output:

```
PROBLEMS 1  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  powershell + - [] [X] ... | [C] [X]

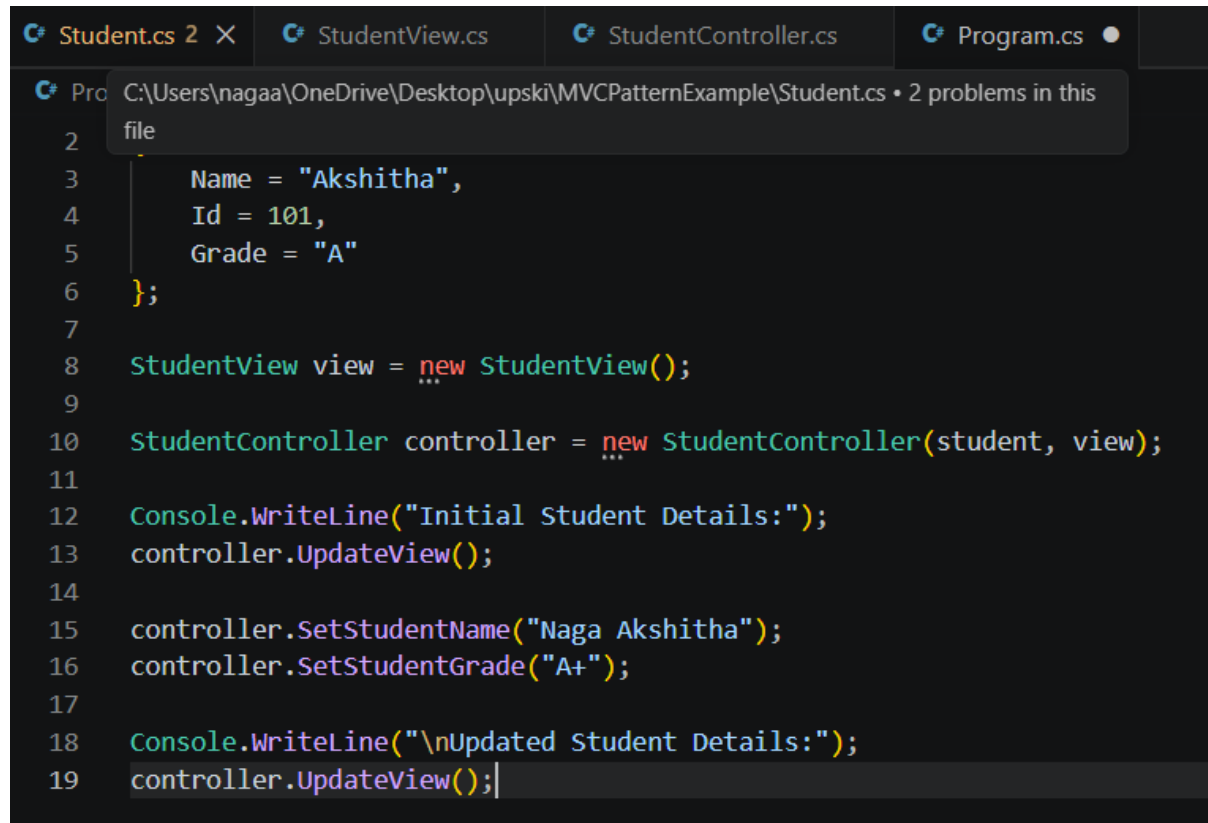
PS C:\Users\nagaa\OneDrive\Desktop\upski\CommandPatternExample> dotnet run
C:\Users\nagaa\OneDrive\Desktop\upski\CommandPatternExample\RemoteControl.cs(3,21): warning CS8618: Non-nullable field 'command' must contain a non-null value when exiting constructor. Consider adding the 'required' modifier or declaring the field as nullable.
Light is ON
Light is OFF
PS C:\Users\nagaa\OneDrive\Desktop\upski\CommandPatternExample> 
```

Exercise 10: Implementing the MVC Pattern

Scenario:

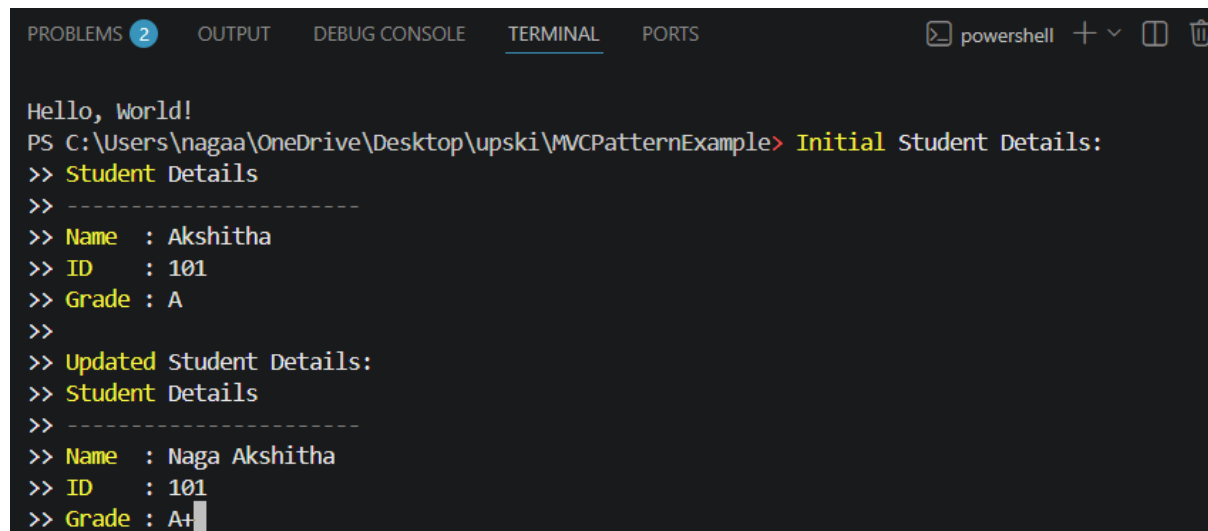
You are developing a simple web application for managing student records using the MVC pattern.

CODE:



```
Student.cs 2 X StudentView.cs StudentController.cs Program.cs
C:\Users\nagaa\OneDrive\Desktop\upski\MVCPatternExample\Student.cs • 2 problems in this
file
2
3     Name = "Akshitha",
4     Id = 101,
5     Grade = "A"
6 };
7
8 StudentView view = new StudentView();
9
10 StudentController controller = new StudentController(student, view);
11
12 Console.WriteLine("Initial Student Details:");
13 controller.UpdateView();
14
15 controller.SetStudentName("Naga Akshitha");
16 controller.SetStudentGrade("A+");
17
18 Console.WriteLine("\nUpdated Student Details:");
19 controller.UpdateView();
```

OUTPUT:



```
PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell + - [ ] [X]
Hello, World!
PS C:\Users\nagaa\OneDrive\Desktop\upski\MVCPatternExample> Initial Student Details:
>> Student Details
>> -----
>> Name : Akshitha
>> ID : 101
>> Grade : A
>>
>> Updated Student Details:
>> Student Details
>> -----
>> Name : Naga Akshitha
>> ID : 101
>> Grade : A+
```

Exercise 11: Implementing Dependency Injection

Scenario:

You are developing a customer management application where the service class depends on a repository class. Use Dependency Injection to manage these dependencies.

Code:

```
erRepository.cs  C# CustomerRepositoryImpl.cs  C# CustomerService.cs  C# Program.cs X ▶
C# Program.cs
1  // Create Repository
2  CustomerRepository repository = new CustomerRepositoryImpl();
3
4  // Inject Repository into Service
5  CustomerService service = new CustomerService(repository);
6
7  // Use Service
8  service.GetCustomer(101);

CustomerRepository.cs  C# CustomerRepositoryImpl.cs  C# CustomerService.cs X  C# Program. ▶
C# CustomerService.cs > CustomerService
3 references
1  public class CustomerService
2  {
3      2 references
4      private CustomerRepository customerRepository;
5
6      // Constructor Injection
7      1 reference
8      public CustomerService(CustomerRepository customerRepository)
9      {
10         this.customerRepository = customerRepository;
11
12         1 reference
13         public void GetCustomer(int id)
14         {
15             string customer = customerRepository.FindCustomerById(id);
16             Console.WriteLine(customer);
17         }
18     }
19 }
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
powershell + - [ ] [ ] ... |
PS C:\Users\nagaa\OneDrive\Desktop\upski\DependencyInjectionExample> dotnet run
Customer Found: ID = 101, Name = Akshitha
PS C:\Users\nagaa\OneDrive\Desktop\upski\DependencyInjectionExample> |
```